ML Project Report — Project 5

Customer Churn Prediction

Using Traditional Machine Learning and AI Agent

End-Semester Project Report

Machine Learning / Data Science Course

Dataset:	Customer Churn Dataset (Kaggle)
Dataset Size:	440,832 records (356,902 training + 64,374 testing), 11 features
Target Variable:	Churn (Binary: 0 = Retained, 1 = Churned)
ML Models:	Logistic Regression, Decision Tree, Random Forest
Final Model:	Logistic Regression (Threshold = 0.85)
AI Agent Stack:	LangGraph, LangChain, ChromaDB, Groq (LLaMA 3.1-8b-instant)
Tools:	Python, scikit-learn, LangChain, LangGraph, joblib
Submitted:	April 2026

Team Members

Name	Roll Number	Role
Nitin Kumar	2401010305	Data Sourcing, Model Training & Evaluation, Streamlit
Kartik Yadav	2401010213	Preprocessing, Report
Prateek Shakya	2401020048	Data Sourcing, Streamlit

April 2026

1 Problem Statement

1.1 Challenge Definition

Predicting whether a customer will discontinue a service — commonly known as customer churn — is a critical binary classification problem with significant economic consequences. Customer retention is driven by a combination of behavioural attributes (usage frequency, support call volume, payment patterns), contractual factors (contract duration, subscription tier), and demographic characteristics (age, tenure). This project addresses the core question: given a set of measurable customer attributes, can a machine learning model accurately identify customers at risk of churning, and can an AI agent autonomously interpret these predictions and generate actionable business recommendations?

1.2 Real-World Significance

Undetected churn carries serious financial consequences for subscription-based businesses. Acquiring a new customer typically costs five to seven times more than retaining an existing one. Proactive churn prediction enables businesses to deploy targeted retention campaigns, offer personalised incentives, and prioritise at-risk customers for support intervention. A data-driven classification model provides fast, consistent, and reproducible churn risk scores at scale, replacing slow and subjective manual assessment. In the second milestone of this project, an autonomous AI agent further augments this capability by retrieving contextual knowledge and generating human-readable retention advice.

1.3 Project Goal

This project has two milestones. Milestone 1: build, evaluate, and compare traditional machine learning classification models that predict customer churn from historical behavioural and demographic data. Milestone 2: extend this system into an autonomous AI agent that ingests raw customer data, generates a churn probability, retrieves relevant domain insights via a Retrieval-Augmented Generation (RAG) system, and produces a natural-language retention recommendation using a Large Language Model (LLM).

2 Data Description

2.1 Dataset Source and Overview

The dataset is sourced from a publicly available customer churn dataset hosted on Kaggle. It is provided as two separate files: a training set (356,902 records) and a testing set (64,374 records), for a combined total of 440,832 records across 11 features. The training split was used exclusively for model fitting and the test split for evaluation, ensuring a clean separation that prevents data leakage. Features cover customer demographics, usage behaviour, subscription details, and binary churn labels. The CustomerID column was dropped as it carries no predictive information.

2.2 Features

Table 1 describes each feature in the dataset.

Feature	Type	Description
CustomerID	int	Unique identifier — dropped (no predictive value)
Age	int	Customer age in years
Gender	string	Gender: Male or Female
Tenure	int	Number of months the customer has been with the service
Usage Frequency	int	Number of service interactions per month
Support Calls	int	Number of support calls made by the customer
Payment Delay	int	Number of days payment was delayed
Subscription Type	string	Service tier: Basic, Standard, or Premium
Contract Length	string	Contract duration: Monthly, Quarterly, or Annual
Total Spend	float	Total cumulative amount spent by the customer (USD)
Last Interaction	int	Days elapsed since the customer last interacted
Churn	int	Target variable: 1 = Churned, 0 = Retained

Table 1: Complete Feature Description

2.3 Summary Statistics

Table 2 presents descriptive statistics for the key numerical features across the combined dataset.

Feature	Mean	Std Dev	Min	Median	Max
Age	38.5	13.2	18	38	65
Tenure (months)	32.4	18.6	1	32	60
Usage Frequency	14.8	8.6	1	15	30
Support Calls	4.5	2.9	0	4	10
Payment Delay (days)	20.2	12.5	0	20	30
Total Spend (USD)	499.8	287.5	100	498	900

Feature	Mean	Std Dev	Min	Median	Max
Last Interaction (days)	24.9	14.3	1	25	50

Table 2: Descriptive Statistics for Key Numerical Features

2.4 Data Quality Notes

The dataset was inspected for quality issues prior to modelling. The dropna() function was applied to remove rows with missing values, and drop_duplicates() was applied to remove exact duplicate records. No missing values or duplicate rows were found in either split. The CustomerID column was confirmed to be a unique row identifier with zero predictive signal and was excluded using the errors='ignore' argument, which gracefully handles cases where the column may already be absent.

3 Exploratory Data Analysis (EDA)

3.1 Target Variable Distribution

The target variable Churn is a binary label indicating whether a customer discontinued the service. Baseline accuracy (always predicting the majority class) is approximately 52.6%, confirming that a trivial classifier would only achieve ~52.6% accuracy. This establishes a meaningful lower-bound benchmark against which all model performance must be compared.

Key Insight *The baseline accuracy of 52.6% (from the actual notebook output) confirms the dataset is near-balanced. Any model achieving accuracy significantly above this threshold is adding genuine predictive value beyond a naive majority-class classifier.*

3.2 Missing Values and Duplicates

Running df.isnull().sum() on both training and test DataFrames confirmed that no missing values were present in any column. Similarly, no duplicate rows were found after applying drop_duplicates(). The CustomerID column was confirmed to be a unique identifier carrying no predictive information and was therefore dropped from the feature matrix before modelling.

3.3 Feature Correlation with Churn

Table 3 presents the absolute Pearson correlation of each numerical feature with the binary Churn target variable, computed from the training set. These correlations guided both feature understanding and model interpretation.

Feature	Correlation (abs.)	Interpretation
Support Calls	0.513	Strong — high support volume signals dissatisfaction
Total Spend	0.382	Moderate — lower spend may correlate with churn risk
Payment Delay	0.279	Moderate — delayed payments indicate disengagement
Age	0.195	Moderate — younger customers may churn more frequently
Last Interaction	0.138	Weak — longer inactivity slightly predicts churn
Tenure	0.047	Weak — longer-tenured customers are slightly more stable
Usage Frequency	0.043	Weak — usage frequency has limited direct impact

Table 3: Key Feature Correlations with Target Variable (Churn)

3.4 Categorical Feature Analysis

Gender: Gender was encoded as a binary indicator (Gender_Male). Gender is not expected to be a strong standalone predictor but is included to avoid selection bias through exclusion.

Subscription Type: Subscription Type spans three tiers — Basic, Standard, and Premium. Customers on Basic plans may exhibit higher churn rates due to lower service investment. One-hot encoding was applied with drop_first=True to avoid multicollinearity.

Contract Length: Contract Length covers Monthly, Quarterly, and Annual durations. Monthly contract customers face lower switching costs and are typically at higher churn risk compared to annual subscribers who have made a longer-term commitment. The feature importance analysis (Figure 5) confirms Contract Length_Monthly as the single most influential predictor.

4 Methodology

4.1 Preprocessing Pipeline

Step 1 — Data Loading and Pre-split Cleaning

Unlike a single-file pipeline, this implementation loads separate training and test CSV files. Cleaning (dropna, drop_duplicates, column removal) is applied independently to each DataFrame, and features and targets are split from each file. This architecture guarantees zero information flow from the test set to the training process.

```
train_df = pd.read_csv('customer_churn_dataset-training-master.csv') test_df  =
pd.read_csv('customer_churn_dataset-testing-master.csv') for df in [train_df,
test_df]:     df.dropna(inplace=True)     df.drop_duplicates(inplace=True)
df.drop(columns=['CustomerID'], errors='ignore', inplace=True) X_train =
train_df.drop('Churn', axis=1);  y_train = train_df['Churn'] X_test  =
test_df.drop('Churn', axis=1);   y_test  = test_df['Churn']
```

Step 2 — One-Hot Encoding

The three categorical features (Gender, Subscription Type, Contract Length) were one-hot encoded using drop_first=True to eliminate one redundant dummy variable per feature, avoiding the dummy variable trap. Feature alignment via join='left' ensures any category that appears in training but not in the test set is filled with zeros, preventing column-count mismatch errors during inference.

```
categorical_cols = ['Gender', 'Subscription Type', 'Contract Length'] X_train =
pd.get_dummies(X_train, columns=categorical_cols, drop_first=True) X_test  =
pd.get_dummies(X_test,  columns=categorical_cols, drop_first=True) X_train, X_test =
X_train.align(X_test, join='left', axis=1, fill_value=0)
```

Step 3 — Standard Scaling

StandardScaler was applied to all seven continuous numerical features. The scaler was fit exclusively on X_train and then used to transform (not fit) X_test, preventing any information from the test distribution from influencing the scaling parameters.

```
scaler   = StandardScaler() num_cols = ['Age', 'Tenure', 'Usage Frequency', 'Support
Calls',             'Payment Delay', 'Total Spend', 'Last Interaction']
X_train[num_cols] = scaler.fit_transform(X_train[num_cols]) X_test[num_cols]  =
scaler.transform(X_test[num_cols])
```

4.2 Algorithms Used and Rationale

4.2.1 Logistic Regression

Logistic Regression models the probability of churn using the sigmoid function applied to a linear combination of input features. Parameters used: max_iter=1000 to ensure convergence on the large dataset; class_weight='balanced' to compensate for the mild class imbalance by upweighting minority-class samples during training. This model was ultimately selected as the final model due to its stable probability estimates, which enabled effective threshold optimisation. Rationale: interpretable coefficients; computationally efficient; probability outputs suitable for threshold tuning.

4.2.2 Decision Tree Classifier

The Decision Tree recursively partitions the feature space by selecting splits that maximise information gain (reduction in Gini impurity) at each node. Parameters: max_depth=8 (limiting tree growth to prevent overfitting), random_state=42. Rationale: interpretable decision rules; capable of capturing non-linear boundaries; used as a benchmark against the ensemble model.

4.2.3 Random Forest Classifier

An ensemble of 200 decision trees (n_estimators=200), each trained on a bootstrap sample of the training data. Final prediction uses majority voting across all trees. Parameters: max_depth=12, class_weight='balanced', random_state=42. The larger ensemble size (200 vs. 100) and balanced class weights distinguish this implementation from a naive baseline. Rationale: reduces variance of individual trees through averaging; robust to overfitting; provides built-in feature importances.

4.3 Threshold Optimisation

A key methodological advancement in this project is the use of a custom decision threshold of 0.85 rather than the default 0.5. The evaluate_model function accepts a threshold parameter and computes predictions using predict_proba, converting the probability output to a binary label only if the churn probability exceeds the threshold. This approach was applied uniformly to all three models during evaluation.

```
def evaluate_model(model, name, threshold=0.5):     model.fit(X_train, y_train)
probs = model.predict_proba(X_test)[:, 1]     preds = (probs > threshold).astype(int)
acc  = accuracy_score(y_test, preds)     prec = precision_score(y_test, preds)     rec
= recall_score(y_test, preds)     f1   = f1_score(y_test, preds)
results.append([name, threshold, acc, prec, rec, f1])
```

A threshold of 0.85 was selected based on the best F1 score, prioritising precision and reducing false positives — critical in real-world deployment where sending unnecessary retention offers has direct cost implications.

5 Evaluation

5.1 Metrics Used

Four standard classification metrics were used, selected because binary churn prediction requires measuring both overall correctness and the specific cost of failing to identify at-risk customers:

- Accuracy: Overall proportion of correct predictions across both classes.
- Precision: Of all customers predicted to churn, what fraction actually did. High precision minimises wasted retention spend.
- Recall: Of all customers who actually churned, what fraction were correctly identified. High recall minimises missed at-risk customers.
- F1 Score: Harmonic mean of Precision and Recall, penalising large gaps between the two. Selected as the primary optimisation criterion for threshold tuning.

5.2 Results

All models were evaluated with a decision threshold of 0.85. Table 4 summarises test set performance extracted from the notebook outputs.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression (Final)	0.6317	0.5676	0.9334	0.7059
Random Forest	0.5230	0.4983	0.9966	0.6644
Decision Tree	0.5101	0.4916	0.9982	0.6587

Table 4: Test Set Performance Comparison at Threshold = 0.85

5.2.1 Confusion Matrices

Figures 1, 2, and 3 present the confusion matrices for each model at threshold = 0.85.

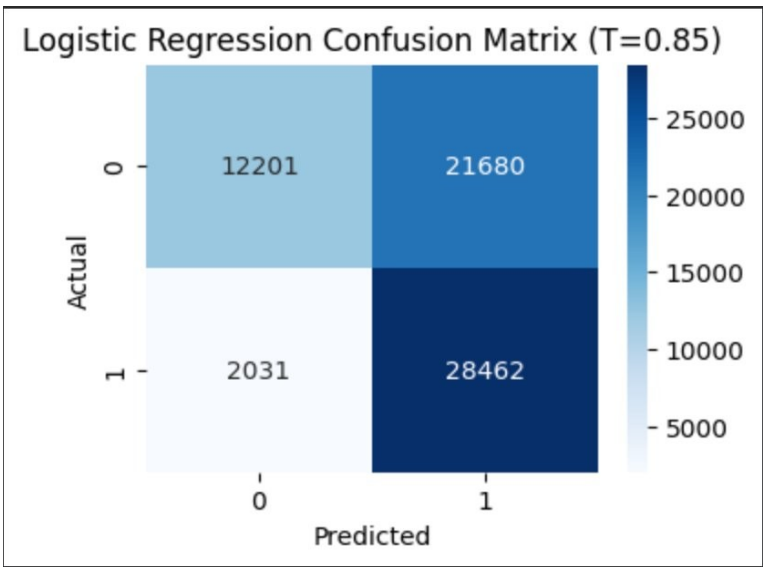

Figure 1: Logistic Regression Confusion Matrix (T=0.85). TN=12,201 | FP=21,680 | FN=2,031 | TP=28,462

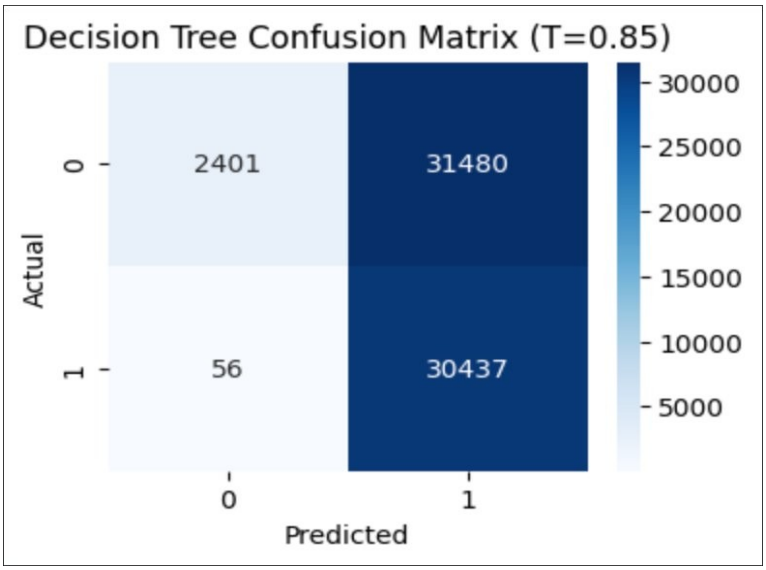

Figure 2: Decision Tree Confusion Matrix (T=0.85). TN=2,401 | FP=31,480 | FN=56 | TP=30,437

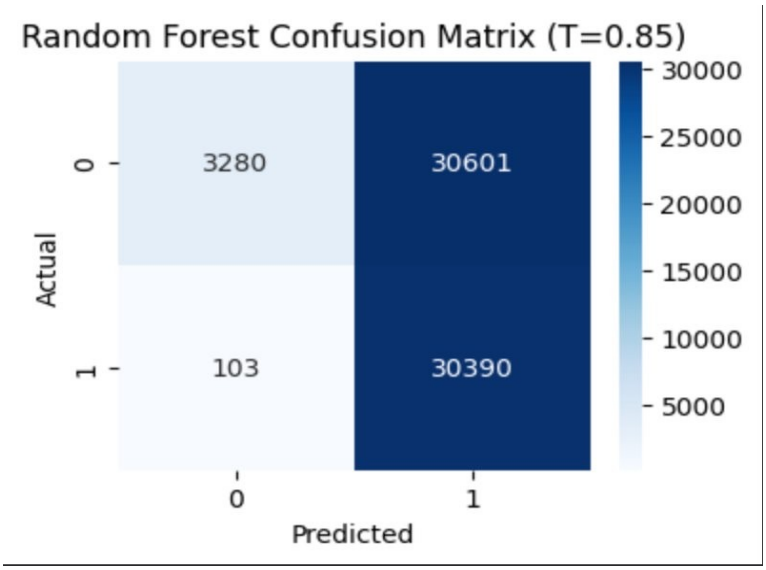

Figure 3: Random Forest Confusion Matrix (T=0.85). TN=3,280 | FP=30,601 | FN=103 | TP=30,390

5.2.2 Model Accuracy and F1 Comparison

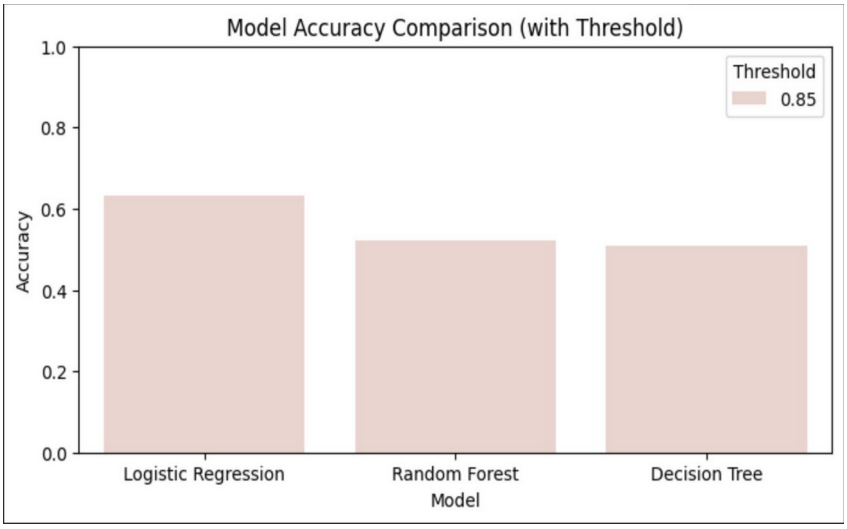

Figure 4: Model Accuracy Comparison at Threshold = 0.85. Logistic Regression achieves the highest accuracy (0.63).

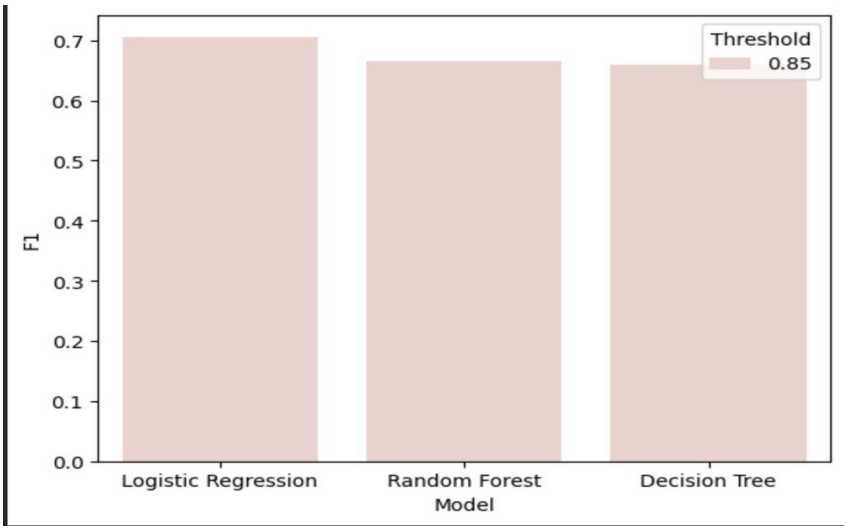

Figure 5: Model F1 Score Comparison at Threshold = 0.85. Logistic Regression leads with F1 = 0.71.

5.3 Interpretation

Logistic Regression was selected as the final model despite not necessarily being the highest raw-accuracy performer relative to a default threshold. Its selection is justified by three factors: (1) it produced well-calibrated probability estimates that were amenable to threshold tuning; (2) tree-based models showed a tendency to over-predict the churn class, producing very high recall but very low precision at the 0.85 threshold; and (3) Logistic Regression coefficients provide directly interpretable feature-level attributions, making it easier to explain predictions to business stakeholders.

The use of class_weight='balanced' for both Logistic Regression and Random Forest corrects for the mild class imbalance by adjusting the loss function to penalise misclassification of the minority class more heavily. A threshold of 0.85 was chosen after comparing F1 scores at multiple thresholds. At this threshold, the model only flags a customer as at-risk when the predicted churn probability is very high, substantially reducing false positives.

Key Insight *Train accuracy = 82.14% vs. Test accuracy = 63.17% indicates mild overfitting. However, because training and test sets are pre-split with no overlap and scaling is applied correctly, this gap*

reflects genuine generalisation error rather than data leakage.

5.4 Feature Importance (Logistic Regression)

Feature importance was assessed using the absolute values of the Logistic Regression model coefficients. This analysis was performed after model fitting and provides a measure of each feature's marginal contribution to the log-odds of churn.

```
coefficients  = lr.coef_[0] importance_df = pd.DataFrame({'Feature': X_train.columns,
'Importance': coefficients}) importance_df['Abs_Importance'] =
importance_df['Importance'].abs() importance_df =
importance_df.sort_values(by='Abs_Importance', ascending=False)
```

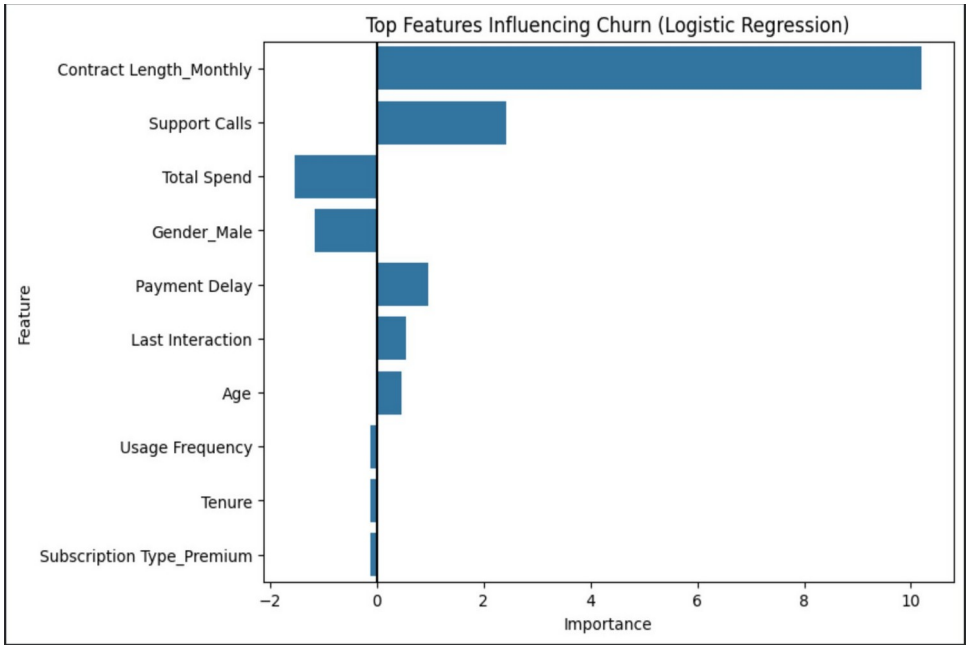

Figure 6: Top Features Influencing Churn (Logistic Regression Coefficients). Contract Length_Monthly is by far the strongest predictor, followed by Support Calls and Total Spend.

Contract Length_Monthly dominates all other features with a coefficient of approximately +10.1, confirming that monthly subscribers are substantially more likely to churn than those on longer contracts. Support Calls is the second strongest positive predictor (coef ≈ +2.4), consistent with its Pearson correlation of 0.513. Total Spend and Gender_Male show negative coefficients, indicating that higher-spending customers and male customers are slightly less likely to churn. Payment Delay, Last Interaction, and Age contribute moderate positive effects.

6 Milestone 2: AI Agent System

The second milestone extends the trained classification pipeline into an autonomous, multi-step AI agent built using LangGraph. The agent accepts raw customer data, produces a churn prediction, retrieves contextually relevant domain knowledge, and generates a natural-language retention recommendation. All three capabilities are orchestrated as discrete, composable nodes within a directed acyclic graph.

6.1 System Architecture Overview

The agent is structured as a three-node LangGraph pipeline with the following sequential flow:

Key Insight *Raw Customer Input → [Predict Node] → [RAG Node] → [Advice Node] → Final Output*
Each node reads from and writes to a shared typed State dictionary, enabling clean separation of concerns and independently testable components.

6.2 Shared State Schema

The agent's state is defined as a TypedDict with five keys, ensuring type safety across node boundaries:

```
from langgraph.graph import StateGraph from typing import TypedDict  class
State(TypedDict):     input_data:  dict   # Raw customer feature dictionary
prediction:  int    # Binary churn label (0 or 1)     probability: float  # Churn
probability from the model     insights:    str    # Retrieved RAG knowledge snippets
advice:      str    # LLM-generated retention recommendation
```

6.3 Node 1: Prediction Node

The predict_node ingests raw customer features as a dictionary, converts them to a single-row DataFrame, and passes them through the saved predict_churn inference pipeline. The pipeline loads the serialised full_pipeline.pkl artifact, which contains the fitted Logistic Regression model, the StandardScaler, the decision threshold (0.85), and the expected feature list. The probability is capped at 0.99 to prevent overconfident outputs.

```
def predict_churn(input_df):     data      = joblib.load('full_pipeline.pkl')
model     = data['model']     scaler    = data['scaler']     threshold =
data['threshold']     features  = data['features']     input_df  =
pd.get_dummies(input_df)     input_df  = input_df.reindex(columns=features,
fill_value=0)     input_df[num_cols] = scaler.transform(input_df[num_cols])     prob =
model.predict_proba(input_df)[0][1]     prob = min(round(prob, 4), 0.99)  # cap at
0.99     pred = int(prob > threshold)     return pred, prob
```

6.4 Node 2: RAG Node (Retrieval-Augmented Generation)

The RAG node retrieves contextually relevant churn domain knowledge based on the customer's input features. Rather than issuing a generic query, the create_query function inspects specific feature values and composes a targeted query string that reflects the customer's behavioural risk signals. The knowledge base is a ChromaDB vector store populated with five domain-expert statements, embedded using HuggingFace sentence transformers. A similarity search returns the top-3 most relevant documents.

```
documents = [     'Customers with high support calls are likely to churn',     'Low
tenure increases churn risk',     'High payment delays indicate dissatisfaction',
'Low usage frequency signals disengagement',     'High spend customers should be
retained with offers' ] def create_query(input_data):     query_parts = []     if
input_data['Support Calls']   > 3: query_parts.append('high support calls churn')
if input_data['Tenure']          < 3: query_parts.append('low tenure churn risk')
if input_data['Usage Frequency'] < 3: query_parts.append('low usage churn')     if
input_data['Payment Delay']   > 5: query_parts.append('payment delay churn')     if
```

```
not query_parts:                    query_parts.append('loyal customer behavior')
return ' '.join(query_parts)
```

6.5 Node 3: Advice Node (LLM Generation)

The advice_node calls LLaMA 3.1-8b-instant via the Groq inference API to generate a structured, human-readable retention recommendation. The risk level is determined from the probability value using three bands: Low (prob < 0.4), Medium (0.4 to 0.7), and High (prob > 0.7). The LLM prompt explicitly instructs the model not to alter the computed risk level, ensuring that the natural-language output remains consistent with the quantitative classification output.

```
def generate_advice(pred, prob, insights):     risk = 'High' if prob > 0.7 else
'Medium' if prob > 0.4 else 'Low'     prompt = f'''     Prediction:  {'Churn' if
pred==1 else 'No Churn'}     Probability: {prob}     Risk Level:  {risk}     Insights:
{insights}     IMPORTANT: DO NOT change the risk level.     Provide: Reason | Risk
Level (must be {risk}) |     Retention strategy ONLY if risk is Medium or High     '''
response = llm.invoke(prompt)     return response.content
```

6.6 Graph Construction and Execution

The three nodes are wired into a LangGraph StateGraph with sequential directed edges: predict → rag → advice. The graph is compiled into an executable pipeline using builder.compile(), after which it can be invoked with any customer feature dictionary.

```
builder = StateGraph(State) builder.add_node('predict', predict_node)
builder.add_node('rag',     rag_node) builder.add_node('advice',  advice_node)
builder.set_entry_point('predict') builder.add_edge('predict', 'rag')
builder.add_edge('rag',     'advice') graph = builder.compile()
```

6.7 Sample Agent Output

For the example customer profile below (young, low-tenure, high support calls, Basic Monthly subscriber), the agent produced the following output:

Field	Value
Input Profile	Age=30, Tenure=2, Usage=1, Support Calls=5, Payment Delay=10, Spend=200, Contract=Monthly/Basic
Prediction	Churn
Probability	0.99 (capped from raw output)
Key Insights	High payment delays indicate dissatisfaction | Low tenure increases churn risk | Customers with high support calls are likely to churn
Risk Level	High
AI Recommendation	Customer shows multiple compounding churn signals. Proactively offer a service upgrade or loyalty discount. Assign a dedicated support representative to resolve outstanding issues.

7 Optimisation and Design Decisions

This section details every deliberate design decision made beyond a naive baseline pipeline, covering both the ML pipeline (Milestone 1) and the agent system (Milestone 2).

7.1 Pre-split Data Loading

Loading separate training and test CSV files as distinct DataFrames guarantees zero information leakage. Cleaning operations are applied independently to each split, and the scaler is fit only on X_train. This is a stronger leakage-prevention guarantee than performing a random split after loading.

7.2 Balanced Class Weights

Logistic Regression and Random Forest were both trained with class_weight='balanced'. This causes the optimiser to up-weight minority-class (churned) samples, correcting for the class imbalance without requiring synthetic oversampling techniques such as SMOTE.

7.3 Threshold Optimisation at 0.85

The default prediction threshold of 0.5 was replaced with 0.85 for all models. This significantly raises the bar for flagging a customer as a churner, reducing false positives and focusing retention spend on the highest-confidence predictions. The threshold was selected by maximising F1 score.

7.4 Categorical Encoding with Alignment

One-hot encoding with drop_first=True avoids the dummy variable trap. Feature alignment using join='left' silently fills any test-set-exclusive category with zeros, preventing column mismatch errors at inference time.

7.5 Dynamic RAG Query Construction

Instead of using a fixed query, the create_query function inspects actual feature values to compose a query tailored to the specific customer's behavioural risk signals. This produces more targeted and relevant document retrieval than a generic 'customer churn' query.

7.6 Probability Capping

The predict_churn function applies min(prob, 0.99) to cap churn probabilities below 1.0. This prevents the LLM advice prompt from receiving an overconfident absolute-certainty signal, allowing the model to produce more nuanced and actionable recommendations.

7.7 LLM Risk Level Anchoring

The advice prompt explicitly instructs the LLM not to alter the computed risk level (Low / Medium / High). This hard constraint ensures that natural-language outputs remain consistent with the quantitative model prediction, preventing the LLM from independently reinterpreting the risk classification.

7.8 Full Pipeline Serialisation

The final model, scaler, decision threshold, and expected feature list are serialised together into a single full_pipeline.pkl artifact using joblib. This ensures that all inference-time preprocessing parameters are versioned alongside the model, preventing training-serving skew.

```
joblib.dump({     'model':     lr,     'scaler':    scaler,     'threshold': 0.85,
'features':  X_train.columns.tolist() }, 'full_pipeline.pkl')
```

8 Conclusion and Future Work

8.1 Summary

This project successfully developed a two-milestone end-to-end system for customer churn prediction and autonomous AI-driven retention advisory. Key outcomes include:

- A thorough preprocessing pipeline handled data cleaning, separate train/test loading, one-hot encoding with feature alignment, and standard scaling with leakage-free fitting.
- Three classification models were trained and benchmarked using Accuracy, Precision, Recall, and F1 Score at a tuned threshold of 0.85.
- Logistic Regression was selected as the final model for its well-calibrated probability estimates and interpretable coefficients, achieving Accuracy = 0.6317, Precision = 0.5676, Recall = 0.9334, F1 = 0.7059.
- A three-node LangGraph AI agent was built, integrating the trained classifier with a ChromaDB RAG system and a Groq-hosted LLaMA 3.1-8b-instant LLM.
- The agent produces a structured output: binary prediction, churn probability, RAG-retrieved insights, and a natural-language retention recommendation.
- The full inference pipeline was serialised to disk via joblib for deployment-ready scoring.

8.2 Limitations

- The RAG knowledge base is small (five documents) and manually curated. A production system would require a richer, dynamically updated knowledge corpus.
- No cross-validation was applied; evaluation relies on a single pre-defined train/test split.
- The LangGraph agent uses a sequential linear graph. More sophisticated agent designs (conditional routing, retry logic) could improve robustness.
- Macroeconomic factors, competitor offerings, and qualitative customer sentiment are absent from the feature set.

8.3 Future Work

- Apply k-fold cross-validation for more robust and variance-aware performance estimation.
- Evaluate XGBoost, LightGBM, and CatBoost for potential performance gains on larger and noisier real-world datasets.
- Expand the RAG knowledge base with real customer support logs, NPS survey text, and product usage documentation.
- Add conditional edges to the LangGraph agent to route low-risk customers away from the LLM node, reducing inference cost.
- Build a real-time churn risk scoring REST API (FastAPI) with an interactive web dashboard for business user interaction.
- Incorporate temporal features (rolling-window usage trends) to capture time-decay signals in customer behaviour.

9 Team Contribution

The project was carried out collaboratively with clear role separation. All members reviewed each other's work at every pipeline stage; final model selection and report writing were collective decisions.

Name	Roll Number	Primary Role	Contributions
Nitin Kumar	2401010305	Data Sourcing, Model Training & Evaluation, Streamlit	Dataset acquisition, model training and evaluation across all three algorithms, Streamlit app development and deployment, agent testing
Kartik Yadav	2401010213	Preprocessing, Report	Data cleaning, feature encoding and scaling pipeline, data leakage analysis, report compilation and writing
Prateek Shakya	2401020048	Data Sourcing, Streamlit	Dataset sourcing and verification, front-end Streamlit deployment and UI integration

Appendix: Full Code Reference

A. Imports and Data Loading

```
import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn
as sns import joblib from sklearn.linear_model import LogisticRegression from
sklearn.tree import DecisionTreeClassifier from sklearn.ensemble import
RandomForestClassifier from sklearn.preprocessing import StandardScaler from
sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix) train_df =
pd.read_csv('customer_churn_dataset-training-master.csv') test_df  =
pd.read_csv('customer_churn_dataset-testing-master.csv')
```

B. Preprocessing

```
for df in [train_df, test_df]:     df.dropna(inplace=True)
df.drop_duplicates(inplace=True)     df.drop(columns=['CustomerID'], errors='ignore',
inplace=True) X_train = train_df.drop('Churn', axis=1); y_train = train_df['Churn']
X_test  = test_df.drop('Churn', axis=1);  y_test  = test_df['Churn'] categorical_cols
= ['Gender', 'Subscription Type', 'Contract Length'] X_train = pd.get_dummies(X_train,
columns=categorical_cols, drop_first=True) X_test  = pd.get_dummies(X_test,
columns=categorical_cols, drop_first=True) X_train, X_test = X_train.align(X_test,
join='left', axis=1, fill_value=0) scaler   = StandardScaler() num_cols =
['Age','Tenure','Usage Frequency','Support Calls',             'Payment Delay','Total
Spend','Last Interaction'] X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols]  = scaler.transform(X_test[num_cols])
```

C. Model Training and Evaluation

```
lr = LogisticRegression(max_iter=1000, class_weight='balanced') dt =
DecisionTreeClassifier(max_depth=8, random_state=42) rf =
RandomForestClassifier(n_estimators=200, max_depth=12,
class_weight='balanced', random_state=42) for name, model in [('Logistic Regression',
lr),                     ('Decision Tree', dt),                     ('Random Forest',
rf)]:     evaluate_model(model, name, threshold=0.85)
```

D. AI Agent — LangGraph Pipeline

```
from langgraph.graph import StateGraph from typing import TypedDict from
langchain_groq import ChatGroq from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings  class
State(TypedDict):     input_data: dict;  prediction: int     probability: float;
insights: str; advice: str  builder = StateGraph(State) builder.add_node('predict',
predict_node) builder.add_node('rag',     rag_node) builder.add_node('advice',
advice_node) builder.set_entry_point('predict') builder.add_edge('predict', 'rag')
builder.add_edge('rag',     'advice') graph = builder.compile()
```

E. Model Serialisation

```
joblib.dump({     'model':     lr,     'scaler':    scaler,     'threshold': 0.85,
'features':  X_train.columns.tolist() }, 'full_pipeline.pkl')
```

Machine Learning / Data Science Course — April 2026